
THE GO CONCURRENCY SURVIVAL KIT

Your Complete Reference for Writing
Performant, Maintainable Concurrent Code

- ✓ The 3 Essential Concurrency Primitives
- ✓ 8 Battle-Tested Concurrency Patterns
- ✓ How to Prevent Goroutine Leaks
- ✓ Fan-Out/Fan-In for 5x+ Performance
- ✓ Common Pitfalls & How to Avoid Them
- ✓ Quick Reference Tables & Decision Guides

MASTER CONCURRENCY. WRITE BETTER GO CODE.

by Georgio Tunson

CONTENTS

02 The Concurrency Primitives

Goroutines — Lightweight Threads

The Fork-Join Model

Channels — Communication Between Goroutines

Select — Handling Multiple Channels

05 Essential Concurrency Patterns

For-Select Loop

Done Channel — Graceful Shutdown and Preventing Goroutine Leaks

Generator — Producing Data On Demand

Pipeline — Chaining Stages Together

Fan-Out/Fan-In — Concurrent Processing

Or-Done Channel — Wrapping Channels for Cancellation

Context Package — Production Cancellation

Confinement

14 Survival Guide

Common Pitfalls Quick Reference

Pitfall Details

Pattern Decision Guide

Golden Rules

BEFORE YOU BEGIN

This guide is designed to be your go-to reference for Go concurrency. Whether you're debugging a deadlock at 2am or designing a new concurrent system from scratch, the patterns and principles in these pages will help you write code that's both correct and performant.

WHO THIS IS FOR

Intermediate Go developers who understand the basics of the language but want to level up their concurrency skills. You should be comfortable with functions, structs, and interfaces before diving in.

HOW TO USE THIS GUIDE

Read Part 1 first — the primitives are foundational. Then work through the patterns in Part 2. Keep Part 3 bookmarked for quick reference when you're stuck. The pitfall details section is especially useful when debugging.

ABOUT THE CODE

All code examples are kept intentionally minimal to highlight the pattern, not distract with application-specific details. They're meant to be starting points — adapt them to your needs.

*"Don't communicate by sharing memory;
share memory by communicating."*

— Rob Pike

PART 1: THE CONCURRENCY PRIMITIVES

Go makes concurrency deceptively easy. Slap the **go** keyword in front of a function and boom — you've got a concurrent process. But this simplicity is a double-edged sword. Without understanding what's actually happening under the hood, you'll write code that's riddled with **race conditions**, **deadlocks**, and **memory leaks**. So let's build that understanding. There are three primitives that make Go's concurrency work: **goroutines**, **channels**, and **select**. Once you truly grasp these, everything else — the patterns, the best practices, the debugging — clicks into place.

1. Goroutines — Lightweight Threads

Place the **go** keyword in front of any function call and it becomes a **goroutine** — a lightweight thread managed by the Go runtime. Here's the critical thing to understand: when main spawns a goroutine, it doesn't wait around. Main continues executing immediately while the goroutine runs off and does its thing in the background. This is what makes concurrency possible — main can spawn dozens of goroutines and they'll all make progress independently.

Main itself is also a goroutine. When your program starts, the Go runtime creates the main goroutine to run your `main()` function. Every goroutine you spawn with the **go** keyword is a child of this main goroutine. That's why when main finishes, your program exits — the Go runtime shuts down and takes all of your goroutines with it, whether they've finished their work or not.

```
func main() {  
    go myFunction() // Forks off, main continues immediately  
    go anotherFunc() // Forks off, main continues immediately  
  
    // Without synchronization, main might exit before  
    // goroutines finish - we'll fix this next  
    time.Sleep(time.Second)  
}
```

The **go** keyword spawns each function call as a separate goroutine. Main doesn't wait — it immediately moves to the next line. The `time.Sleep` is a crude hack to keep main alive long enough for the goroutines to finish. We'll learn the proper way next.

■■ Goroutines don't wait! If main finishes before your goroutines, you'll never see their results.

So if main doesn't wait, how do we ever get results back from our goroutines? This can be better understood by first understanding Go's model of concurrency known as the **fork-join model**.

DEEP DIVE

The Fork-Join Model

When *main* spawns a goroutine, that's the **fork** — the child splits off from the parent. At some point, the child needs to **join** back with *main* so we can collect its results. Without a **join point**, *main* might finish and exit before the goroutines complete, and you'll never see their output. Below, we create this **join point** by making use of a **WaitGroup**. But the **join point** can also be implemented via **Channels** which we will go over in the next section:

```
func main() {
    var wg sync.WaitGroup

    wg.Add(1) // Increment BEFORE spawning
    go func() {
        defer wg.Done() // Decrement when done
        doWork()
    }()

    wg.Wait() // Join point - main blocks here until counter is 0
}
```

2. Channels — Communication Between Goroutines

Now we can spawn goroutines and wait for them to finish. But what if they need to talk to each other? Goroutines run independently — they don't share memory by default, and they have no idea the others even exist. This is where **channels** come in. A channel is a typed conduit that goroutines can send data into and receive data from. The Go runtime handles the synchronization of these sends and receives for you, which means you don't need mutexes or locks to safely pass data between goroutines. This is the essence of Go's concurrency philosophy, captured in Rob Pike's famous advice: *"Do not communicate by sharing memory; instead, share memory by communicating."* In other words, rather than having goroutines access shared variables protected by locks, you pass the data itself through channels. Keep in mind though — if you pass a pointer through a channel, you're still sharing memory. The channel makes the handoff safe, but what you do with that pointer afterward is on you.

```
ch := make(chan int) // Unbuffered (synchronous)
ch := make(chan int, 10) // Buffered (async until full)
ch <- 42 // Send data to channel
value := <-ch // Receive data from channel
close(ch) // Only sender closes!
```

The arrow `<-` always points in the direction data flows. `ch <- 42` sends 42 into the channel. `value := <-ch` receives from the channel. When you're done sending, `close(ch)` signals receivers that no more data is coming.

Channels come in two flavors: **unbuffered** and **buffered**. An unbuffered channel is a direct handoff — a **synchronization point** where the sender **blocks** until a receiver is ready, and they exchange data in that moment. There's no queue, no waiting room. Buffered channels, on the other hand, add capacity: you can send data into the buffer and continue without waiting, at least until the buffer fills up. Once it's full, you block just like an unbuffered channel until someone reads from the other end.

3. Select — Handling Multiple Channels

What happens when a goroutine needs to listen to multiple channels at once? You can't just read from one and hope for the best — you might miss messages on the others. The **select** statement solves this. It waits on multiple channel operations simultaneously and executes whichever one is ready first. If multiple channels are ready at the same time, it picks one **at random** to prevent starvation.

```
select {
case msg := <-ch1:
    fmt.Println("Received from ch1:", msg)
case msg := <-ch2:
    fmt.Println("Received from ch2:", msg)
default:
    fmt.Println("No messages ready")
}
```

Without a **default** case, select blocks until one of the channel cases is ready. With a **default** case, select becomes non-blocking — if no channels are ready, it immediately executes the default case instead of waiting.

One important thing to understand: a select statement executes exactly once. It picks one case (or default), runs it, and then execution continues to the next line after the select block. If you want to continuously listen to channels, you need to wrap the select in a loop. We will explore this in the next section on patterns.

That's it — **goroutines**, **channels**, and **select**. These three primitives are the foundation of everything in Go concurrency. Goroutines give you lightweight concurrent execution. Channels let those goroutines communicate safely. And select lets a single goroutine coordinate multiple channels at once. Every pattern you'll see in the next section is just a clever combination of these three building blocks.

PART 2: ESSENTIAL CONCURRENCY PATTERNS

With the primitives under your belt, you're ready for patterns — reusable solutions to common concurrency problems. These eight patterns will handle the vast majority of concurrent code you'll ever write. Master these, and you'll have a solid foundation to tackle anything else that comes your way. And the best part? Each pattern is just some combination of the primitives that you've just learned! So you are already half way there.

Pattern 1: For-Select Loop

Remember when we went over the fundamental Go concurrency primitives in the previous section and talked about the select statement? We mentioned that it executes exactly once. Well, that's not very useful on its own since in many situations we need to receive multiple data objects from multiple channels. But how do we actually do that? We do that by making use of our first pattern: the **for-select loop**. This pattern is simple — we wrap the select statement in an infinite loop so it repeats indefinitely. But since this loop runs forever, we need some way to gracefully shut down or return out of the goroutine at some point. That's why we also read from a done channel, which we'll explain in the next pattern.

```
for {
    select {
        case data := <-dataChan:
            process(data)
        case <-done:
            return
    }
}
```

This loop runs forever, waiting for one of two things: either data arrives on `dataChan` and we process it, or the `done` channel closes and we exit cleanly.

Pattern 2: Done Channel — Graceful Shutdown and Preventing Goroutine Leaks

You'll notice we used a `done` channel in that last example. A done channel is simply a channel that carries no data — its only purpose is to be closed. When a parent goroutine closes it, that closure broadcasts to every child listening, telling them to stop what they're doing and exit. This solves a critical problem: without an exit signal, a goroutine waiting on a channel that never receives data and is never closed will block forever. It doesn't crash, it doesn't timeout, it just sits there — alive but stuck. That's a **goroutine leak**. In long-running programs, these leaked goroutines can pile up, each one holding onto memory and references that can never be freed, which can eventually cause performance degradation or even crashes.

```
done := make(chan struct{})

// Start worker with done channel
go worker(dataChan, done)

// When ready to shut down
close(done) // Broadcasts to ALL listeners
```

We use `chan struct{}` because we don't actually send any data — we just close it. An empty struct takes zero bytes of memory, making it the most efficient choice for a signaling-only channel. Closing a channel broadcasts to every goroutine listening to it, which makes shutdown simple: `close(done)` once, and all your workers exit cleanly.

■ Closing a channel broadcasts to ALL receivers. That's why we `close(done)` instead of sending a value — it wakes up every goroutine listening to it.

Pattern 3: Generator — Producing Data On Demand

A **generator** is a function that returns a channel and spawns a goroutine to produce values on that channel. You call the function, it hands you back a channel immediately, and values appear on that channel as the internal goroutine creates them. The consumer pulls values at whatever pace it wants, and the generator keeps producing until it's told to stop.

```
func generator(done <-chan struct{}) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for {
            select {
            case out <- rand.Intn(100):
            case <-done:
                return
            }
        }
    }()
    return out
}
```

This generator produces random numbers forever until the parent closes `done`. The select statement either sends a new random value or exits when `done` closes.

Pattern 4: Pipeline — Chaining Stages Together

You should start to notice these patterns building on top of each other. The generator we just wrote uses the done channel pattern and the for-select loop from earlier. Now we'll use that generator as the first stage of a **pipeline**. A pipeline breaks complex processing into discrete stages, connected by channels. Each stage receives data from an input channel, transforms it, and sends it to an output channel. You chain them together: the output of one stage becomes the input of the next. Because each stage runs in its own goroutine, they all execute concurrently — while stage 2 processes item A, stage 1 is already working on item B.

```
// Stage 1: Generator (same as above)
func generator(done <-chan struct{}) <-chan int {
    ...
}

// Stage 2: Transform each value
func square(done <-chan struct{}, in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for n := range in {
            select {
            case out <- n * n:
            case <-done:
                return
            }
        }
    }()
    return out
}

func main() {
    done := make(chan struct{})
    defer close(done)

    // Chain: generator -> square -> print
    for result := range square(done, generator(done)) {
        fmt.Println(result)
    }
}
```

Both stages accept the `done` channel so the entire pipeline can be cancelled cleanly. The generator produces random numbers, square transforms them, and when we decide to stop, closing `done` shuts down every stage in the pipeline. Note: this code will actually run forever since we never close `done` — we'll cover how to handle this shortly.

Pattern 5: Fan-Out/Fan-In — Concurrent Processing

Picture this: you've built a pipeline that processes images. The first stage reads file paths from a channel, the second stage loads each image, and the third stage applies some heavy transformation like resizing or filtering. That third stage is slow — we're talking 500ms per image. This causes the whole pipeline to bottleneck at that stage. So how do we fix this? That's where the **fan-out/fan-in** pattern comes into the picture (no pun intended). We spin up multiple workers to handle the slow stage concurrently — this is the **fan-out** portion of the pattern. Then we take all the results from those workers and merge them back into a single channel — this is the **fan-in** portion. The result? If we have N workers, that slow stage now runs roughly N times faster.

Fan-Out

To fan out we will start multiple goroutines that each run the slow stage. Each goroutine returns its own output channel which we store in a slice containing all output channels. So if we start N workers, we end up with N channels.

```
// Fan-out: start multiple workers, each returns a channel
numWorkers := runtime.NumCPU()
workerResultChannels := make([]<-chan int, numWorkers)
for i := 0; i < numWorkers; i++ {
    workerResultChannels[i] = slowStage(done, inputChan)
}
```

Here we start N workers to execute `slowStage` where N is the value returned from `runtime.NumCPU()`. Let's assume that `slowStage` has similar functionality to previous pipeline stages that we've created where it returns an output channel and spins up a goroutine that produces values for that output channel — the only difference being that this stage is particularly slow.

Fan-In

Now we have multiple worker output channels, but we want a single stream of results. This is where the fan-in portion of the pattern comes into play. We will spawn a goroutine for each worker output channel that was produced during the fan-out stage. Those goroutines will consume the values from their corresponding channels and put them onto a single result stream.

```

func fanIn(done <-chan struct{}, channels ...<-chan int) <-chan int
{
    var wg sync.WaitGroup
    merged := make(chan int)
    merge := func(c <-chan int) {
        defer wg.Done()
        for n := range c {
            select {
            case merged <- n:
            case <-done:
                return
            }
        }
    }
    wg.Add(len(channels))
    for _, c := range channels {
        go merge(c)
    }
    go func() {
        wg.Wait()
        close(merged)
    }()

    return merged
}

```

For each input channel, we spawn a goroutine that reads values and forwards them to `merged`. The `WaitGroup` tracks when all input channels are drained, then we close the merged channel.

Bringing It All Together

```

numWorkers := runtime.NumCPU()
workerResultChannels := make([]<-chan int, numWorkers)
for i := 0; i < numWorkers; i++ {
    workerResultChannels[i] = slowStage(done, inputChan)
}

results := fanIn(done, workerResultChannels...)

for result := range results {
    fmt.Println(result)
}

```

Fan-out starts the workers and collects their channels. **Fan-in** merges them into one stream we can range over.

Pattern 6: Or-Done Channel — Wrapping Channels for Cancellation

Picture this: you have a goroutine consuming values from a channel that you don't control. If the parent of your goroutine decides to shut down and signals its children to stop processing via the done channel, your goroutine needs to receive that signal. But it's also busy listening for values from the other channel. The straightforward solution is a for-select loop that listens to both — the channel you're consuming from and the done channel from your parent:

```
func main() {
    done := make(chan struct{})
    defer close(done)

    foreignChan := someLibrary.GetDataStream()

    go func() {
        for {
            select {
                case val, ok := <-foreignChan:
                    if !ok {
                        return
                    }
                    process(val)
                case <-done:
                    return
            }
        }
    }()
}
```

The child goroutine listens for values from `foreignChan` while also checking the `done` channel. When main exits, `defer close(done)` signals the child to stop. If you only need this in one place, this solution is perfectly fine — just use it and move on.

But what if you have a pipeline with five stages, each reading from a different channel? Now you're writing this same for-select loop five times. This **duplication** clutters your code, makes it harder to read, and if you ever need to change the cancellation logic, you have to update it in multiple places. That's when the **or-done** pattern becomes valuable — it lets you write that for-select loop once and reuse it throughout your codebase. It wraps any channel and returns a new one that closes when either the original channel closes OR your done channel closes — whichever happens first.

```

func orDone(done <-chan struct{}, c <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for {
            select {
            case <-done:
                return
            case v, ok := <-c:
                if !ok {
                    return
                }
                select {
                case out <- v:
                case <-done:
                    return
                }
            }
        }
    }()
    return out
}

```

The `orDone` function wraps any channel and returns a new one that respects cancellation. It checks the `done` channel on every receive AND every send. The nested select on the send prevents blocking if `done` closes while we're trying to forward a value.

Usage

```

func main() {
    done := make(chan struct{})
    defer close(done)

    foreignChan := someLibrary.GetDataStream()

    go func() {
        for val := range orDone(done, foreignChan) {
            process(val)
        }
    }()
}

```

Compare this to the for-select version earlier. The child goroutine now uses a clean for-range loop, and the `orDone` wrapper handles the done channel logic. If you have multiple goroutines reading from different channels, they can all reuse this same wrapper.

■ Only use or-done when you have multiple places that need this behavior. For a single use case, the for-select loop is simpler and avoids premature abstraction.

Pattern 7: Context Package — Production Cancellation

The done channel pattern works, but Go's standard library gives us something better for production code: the **context** package. A context carries **deadlines**, **cancellation signals**, and request-scoped values down through your call stack. If used correctly, we can make it so that when you cancel a parent context, all child contexts cancel automatically. This is how you implement timeouts and graceful shutdown in real-world applications.

```
func doWork(ctx context.Context) error {
    select {
    case <-time.After(2 * time.Second):
        return nil // Work completed
    case <-ctx.Done():
        return ctx.Err() // Cancelled or timed out
    }
}

// Create context with 1 second timeout
ctx, cancel := context.WithTimeout(context.Background(), 1*time.Second)
defer cancel() // Always call cancel!

err := doWork(ctx) // Returns "context deadline exceeded"
```

The work takes 2 seconds, but our context times out after 1. When the deadline hits, `ctx.Done()` closes and the select picks that case, returning the timeout error. Always use `defer cancel()` — it releases resources even if the operation completes before the timeout.

Pattern 8: Confinement

What if you could block off a critical section of your code from all outside goroutines, such that only one goroutine has write privileges to that particular section? Since all other goroutines would only have read access to that critical section, you wouldn't need to worry about using mutexes or synchronizing access. This is known as confinement — you are confining the write privileges of that particular section to one goroutine. There are two ways to achieve it: by convention (ad-hoc confinement) or by using your language's scoping rules to enforce it (lexical confinement).

Ad-hoc confinement relies on team discipline — everyone agrees that only certain goroutines will access certain data. But conventions get forgotten, deadlines cause shortcuts, and new team members don't know the unwritten rules. Lexical confinement is stronger because the compiler

enforces it. You structure your code so that unauthorized access is simply not possible.

A common example is a function that creates a channel internally but only exposes a read-only view to the outside world:

```
func generator(done <-chan struct{}) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for {
            select {
            case out <- rand.Intn(100):
            case <-done:
                return
            }
        }
    }()
    return out
}
```

Look familiar? This is the same generator from earlier. Inside the function, we create a channel and spawn a goroutine that writes to it. We return `<-chan int`, a receive-only channel. Here we're confining the *write capability* of the channel to the lexical scope of the goroutine's closure. Only that one goroutine can ever write to or close this channel — the compiler enforces it. The read capability is shared with whoever receives the channel, but the write side is confined to a single concurrent process.

■ You don't have to confine everything. In this example, we only confined the write capability. Confinement is about leveraging your understanding of language features like closures and lexical scope to limit access to certain data or capabilities to a single concurrent process or goroutine.

Wrapping Up

These eight patterns cover the vast majority of concurrent code you'll write in Go. The **for-select loop** and **done channel** give you long-running workers that shut down cleanly. **Generators** produce data on demand, and **pipelines** chain it into streaming processors. **Fan-out/fan-in** handles slow stages concurrently. **Context** handles timeouts and cancellation in production. **Or-done** makes any channel cancellation-aware. And **confinement** leverages lexical scope to restrict access to a single concurrent process. Mix and match these patterns as needed — most real-world concurrent systems combine several of them.

PART 3: SURVIVAL GUIDE

Even experienced Go developers fall into these traps. Below is a quick reference table — use the numbers to jump to detailed explanations with code examples and fixes.

#	Pitfall	Quick Fix
1	The loop variable trap	Pass as function parameter
2	Goroutine leak	Use done channel or context
3	Deadlock	Ensure receiver ready before send
4	Race condition	Use mutex, channels, or confinement
5	Closing nil channel	Always initialize with make()
6	Double close	Sender usually owns closing, use defer
7	Misplaced wg.Add()	Call Add() outside the goroutine

Pitfall Details

#1 ■■ THE LOOP VARIABLE TRAP

■ **Think of it this way:** You're a photographer taking group shots. You tell everyone "smile and hold that pose" — but your camera has a delay. By the time the shutter clicks, people have moved, blinked, or left entirely. You don't capture what you saw when you pressed the button — you capture whatever's there when the camera finally fires.

■ THE BUG

```
for i := 0; i < 5; i++ {  
    go func() {  
        fmt.Println(i)  
    }()  
}  
// Output: 5, 5, 5, 5, 5
```

✓ THE FIX

```
for i := 0; i < 5; i++ {  
    go func(id int) {  
        fmt.Println(id)  
    }(i)  
}  
// Output: 0, 1, 2, 3, 4
```

What's happening: Note: As of Go 1.22, this issue has been fixed — each loop iteration now gets its own copy of the variable. However, if you're working with older codebases or want to understand why you'll see this pattern in existing Go code, read on.

In earlier Go versions, a loop variable like `i` is a single variable that gets overwritten each iteration. The anonymous function inside the goroutine doesn't copy the value of `i` — it captures a reference to `i` itself. Since goroutines don't execute immediately (they're scheduled by the runtime), by the time they actually run and read `i`, the loop has completed and `i` is sitting at its final value of 5. The fix passes `i` as a function argument, which

creates a copy of the value at that exact moment.

■ **KEY INSIGHT:** Passing the loop variable as a function argument creates a copy at the moment of the function call — before the goroutine is scheduled.

#2 ■■ GOROUTINE LEAK

■ **Think of it this way:** You hire a worker and tell them to keep doing a task until you say stop. But you never gave them a radio to receive your request to stop. They keep working forever — even after you've left the building. Meanwhile, the company keeps paying their overtime and they keep using up resources.

■ THE BUG

```
func startWorker() {
    go func() {
        for {
            fmt.Println("running")
        }
    }()
}
```

✓ THE FIX

```
func startWorker(done <-chan struct{})
{
    go func() {
        for {
            select {
            case <-done:
                return
            default:
                fmt.Println("running")
            }
        }
    }()
}
```

What's happening: The goroutine runs an infinite loop with no way to exit. If your program keeps running (say, it's a web server or long-running service), this goroutine will keep running too — consuming CPU and memory forever. Now, if main exits, the Go runtime does shut down all goroutines — but this is an **ungraceful shutdown**. The goroutine gets killed mid-execution with no chance to clean up resources, close connections, or finish writing data. That's why you still need an escape hatch for proper cleanup.

■ **KEY INSIGHT:** Every goroutine needs an escape hatch. Use a done channel or context so the parent can signal "stop what you're doing, we're shutting down."

#3 ■■ DEADLOCK

■ **Think of it this way:** An unbuffered channel is a direct hand-off — like passing a basketball. You can't throw the ball and expect it to hang in mid-air waiting for someone. A receiver must be ready to catch it the moment you throw. If no one's there, you're stuck holding the ball forever.

■ THE BUG

```
ch := make(chan int)
ch <- 42 // Blocked here forever
fmt.Println(<-ch)
```

✓ THE FIX

```
ch := make(chan int)
go func() {
    ch <- 42 // Runs separately
}()
fmt.Println(<-ch) // Can receive
```

What's happening: Line 2 blocks because unbuffered channels require both a sender and receiver to be ready at the same time. But the receiver is on line 3, which can't execute until line 2 completes. You're waiting for yourself — classic deadlock. The fix puts the send in a separate goroutine. Now the goroutine can block on the send while main continues to line 3 and blocks on the receive. Both sides meet up, the hand-off happens, and everyone moves on.

■ **KEY INSIGHT:** With unbuffered channels, the send blocks until a receiver is ready. If both the send and receive are on the same goroutine, you're deadlocked. Put them on separate goroutines. Buffered channels help, but if the buffer is full, you hit the same problem.

#4 ■■ RACE CONDITION

■ **Think of it this way:** Imagine a bank account with \$100. Two tellers process \$50 deposits at the exact same time. Teller A reads the balance: \$100. Teller B also reads the balance: \$100. Teller A adds \$50 and writes \$150. Teller B adds \$50 and writes \$150. You deposited \$100 total, but you only have \$150 — because both tellers read the old balance before either one wrote the new one.

■ THE BUG

```
counter := 0
for i := 0; i < 1000; i++ {
    go func() {
        counter++
    }()
}
// counter = ??? (random!)
```

✓ THE FIX

```
var mu sync.Mutex
counter := 0
for i := 0; i < 1000; i++ {
    go func() {
        mu.Lock()
        counter++
        mu.Unlock()
    }()
}
```

What's happening: `counter++` looks atomic but it's actually three steps: read the value, add one, write the result. If two goroutines both read `counter` as 5, both add 1 to get 6, and both write 6 — you've lost an increment. The fix uses a mutex to create a "locked room" around the operation. Only one goroutine can hold the lock at a time, so each read-add-write completes fully before the next one starts.

■ **KEY INSIGHT:** Any read-then-write operation on shared memory is dangerous with multiple goroutines. Use `sync.Mutex` to ensure only one goroutine accesses the shared data at a time.

#5 ■■ CLOSING NIL CHANNEL

■ **Think of it this way:** Declaring a channel is not the same as creating one. `var ch chan int` creates an empty variable that can hold a channel, but the variable is `nil` — no channel exists yet. You need to actually create the channel using `make()` to fill that variable with a real channel. Without initialization, you're trying to close something that doesn't exist.

■ THE BUG

```
var ch chan int // Declaration only
close(ch)

// PANIC: close of nil channel
```

✓ THE FIX

```
var ch chan int // Declaration
ch = make(chan int) // Initialization
close(ch)
```

What's happening: Declaration (`var ch chan int`) gives you a `nil` reference. Initialization (`ch = make(chan int)`) actually allocates the channel in memory. Calling `close` on a `nil` channel causes a runtime panic because there's nothing to close.

■ **KEY INSIGHT:** Declaration $\neq$ initialization. Always use `make()` to create a channel before using it.

#6 ■■ DOUBLE CLOSE

■ **Think of it this way:** A channel can only be closed once. Attempting to close it again is a programming error — Go assumes you've made a mistake in your logic and panics to alert you. There's no "double close" operation that makes sense, so Go doesn't allow it.

■ THE BUG

```
ch := make(chan int)
close(ch)
close(ch)

// PANIC: close of closed channel
```

✓ THE FIX

```
ch := make(chan int)
defer close(ch) // Safe: closed once
                // when func returns

// ... use the channel ...
```

What's happening: This typically happens when multiple goroutines might close the same channel, or when error handling accidentally closes a channel that was already closed elsewhere. In most cases, the goroutine that writes to a channel should be the one that closes it. But there are exceptions — think carefully about ownership and who is responsible for closing each channel in your code.

■ **KEY INSIGHT:** The goroutine writing to a channel is usually responsible for closing it, but this is not always the case. Think carefully about channel ownership and who is responsible for closing a given channel.

#7 ■■ MISPLACED WG.ADD()

■ **Think of it this way:** A `WaitGroup` is a counter. `Wait()` blocks until the counter reaches zero. If you call `Add(1)` inside the goroutine, there's a race: the main goroutine might reach `Wait()` and see zero before any goroutine has had a chance to increment the counter. It checks, sees nothing to wait for, and continues — leaving your goroutines orphaned.

■ THE BUG

```
var wg sync.WaitGroup
for i := 0; i < 5; i++ {
    go func() {
        wg.Add(1) // Too late!
        defer wg.Done()
        doWork()
    }()
}
wg.Wait() // Returns immediately
```

✓ THE FIX

```
var wg sync.WaitGroup
for i := 0; i < 5; i++ {
    wg.Add(1) // Before spawning
    go func() {
        defer wg.Done()
        doWork()
    }()
}
wg.Wait() // Waits for all 5
```

What's happening: Goroutines don't start immediately — they're scheduled by the runtime. The main goroutine can race ahead and hit `Wait()` before any of the spawned goroutines have even started executing. If `Add(1)` is inside the goroutine, the counter might still be 0 when `Wait()` checks it, so it returns immediately. The fix moves `Add(1)` outside the goroutine, into the main flow. Now the counter is incremented before each goroutine is spawned, guaranteeing the count is correct by the time `Wait()` is called.

■ **KEY INSIGHT:** Always call `Add()` outside the goroutine, in the parent's flow. This guarantees the counter is set before `Wait()` can possibly run.

Which Pattern Should I Use?

Scenario	Pattern
Wait for multiple goroutines	WaitGroup
Process infinite stream	For-Select + Done Channel
Chain data transformations	Pipeline
Slow stage bottleneck	Fan-Out/Fan-In
Cancel after timeout	Context with Timeout
Parent cancels children	Done Channel / Context / <code>ctx.Done()</code>
Limit access to data	Confinement (lexical scoping)

Golden Rules

1. **Every goroutine needs a way to exit** — Use a done channel or context to signal shutdown
2. **The sender usually closes the channel** — But there are exceptions; when deviating from this, think carefully about ownership and channel closure
3. **Pass loop variables as parameters** — Avoids the loop variable trap in Go versions before 1.22
4. **Defer your cleanup** — `defer close(ch)`, `defer wg.Done()`
5. **Use confinement to restrict access where possible** — Leverage lexical scope to ensure only one concurrent process can access certain data
6. **Fan out to eliminate bottlenecks** — When a stage in your pipeline is slow, spin up multiple workers to process concurrently

Final Thoughts

Congratulations on making it through this guide. You've covered the three core building blocks of Go concurrency — **goroutines**, **channels**, and **select** — and learned how they combine to form powerful patterns. You've seen how the **for-select loop** keeps goroutines running indefinitely, how the **done channel** enables graceful shutdown, and how **pipelines** let you chain processing stages together. You've explored **fan-out/fan-in** for handling slow stages concurrently, the **context** package for production-ready cancellation, and **confinement** for restricting access via lexical scope.

More importantly, you've learned the common pitfalls that trip up developers at every level — from the **loop variable trap** to **goroutine leaks** to **race conditions** — and you now know how to avoid them. Concurrency is one of the trickiest areas in programming, but Go's primitives make it approachable. The key is to start simple: use channels for communication, give every goroutine a way to exit, and always test with the race detector (`go run -race main.go`). As you build more concurrent systems, these patterns will become second nature. Keep this guide handy, refer back to it when you're stuck, and don't be afraid to experiment. The best way to truly understand concurrency is to write concurrent code, break it, fix it, and learn from the process.

Sources

- Go Official Blog: go.dev/blog/pipelines
- Rob Pike's "Go Concurrency Patterns" talk: go.dev/talks/2012/concurrency.slide
- "Concurrency in Go" by Katherine Cox-Buday (O'Reilly Media)